

## Table of Contents

### Technical pack for SUDO — Teltonika device and protocol specification

**From:** Kasper Technologies FZ-LLC **Prepared for:** SUDO engineering team **Source:** Teltonika Telematics Wiki (wiki.teltonika-gps.com) — FMC130 pages, Codec specification, GPRS settings. Firmware reference FMB.Ver.03.29.00.

---

#### 1. Hardware

|                | FMC130                                                                            | FMC920                                          |
|----------------|-----------------------------------------------------------------------------------|-------------------------------------------------|
| Role           | Heavy equipment — excavators, cranes, generators, boom lifts, backhoes, forklifts | Light vehicles and trailers — flatbeds, tippers |
| Category       | Advanced Tracker                                                                  | Advanced Tracker                                |
| CAN access     | LVCAN interface, J1939 direct from ECU                                            | OBD-II via Bluetooth dongle                     |
| Network        | LTE Cat 1 with fallback                                                           | LTE Cat 1 with 2G fallback                      |
| Offline buffer | ~422,000 records in internal flash                                                | Internal flash                                  |

Both models are TDRA-approved and deployed on Etisalat/e& and du networks. Pilot 50–100 devices, designing toward 5,000.

---

#### 2. Transport — the critical section

##### 2.1 Available protocols

The device configuration exposes **TCP or UDP only** as the data transport to the record server. There is no MQTT publish path and no HTTPS path in the FMC130 server settings.

This is the basis for our position that **AWS IoT Core cannot terminate these device connections directly**. A protocol gateway is required in front.

##### 2.2 TCP session sequence

Codec 8 uses 0x08 as the codec ID; Codec 8 Extended uses 0x8E.

1. Device opens a TCP connection to the configured host and port.
2. Device sends its **IMEI**: two bytes of length (0x000F, 15 bytes) followed by the IMEI as ASCII bytes. Example: IMEI 356307042441013 → 000F333536333037303432343431303133
3. **Server must reply with a single binary byte** — 0x01 to accept the device, 0x00 to reject it. The device will not proceed until it receives this.
4. Device sends the AVL data packet:

| Prea<br>mble       | Data<br>Field<br>Lengt<br>h | Codec<br>ID        | Numb<br>er of<br>Data<br>1 | AVL<br>Data | Numb<br>er of<br>Data<br>2                  | CRC-1<br>6         |
|--------------------|-----------------------------|--------------------|----------------------------|-------------|---------------------------------------------|--------------------|
| 0x00<br>0000<br>00 | 4<br>bytes                  | 0x08<br>or<br>0x8E | recor<br>d<br>count        | recor<br>ds | must<br>equal<br>Numb<br>er of<br>Data<br>1 | CRC-1<br>6/<br>IBM |

Data Field Length is measured from Codec ID through Number of Data 2. CRC-16/IBM is calculated over the same span.

5. **Server must reply with a 4-byte integer** equal to the number of records accepted. Example: two records received — server returns 00000002.
6. Only on receipt of a valid acknowledgement does the device clear those records from its buffer.

**Implication:** the gateway is stateful and must hold the socket across the full exchange. This is not a per-message stateless invocation pattern.

### 2.3 UDP variant

Codec 8 over UDP adds a packet-ID layer for reliability. The device retries if a valid acknowledgement with the matching AVL Packet ID isn't received within the configured timeout. UDP carries lower data consumption than TCP; the trade-off should be evaluated against SIM data cost at fleet scale.

### 2.4 Acknowledgement mode is configurable

The **ACK Type** setting determines how the device takes confirmation:

- **TCP/IP** — confirmation is taken from the TCP layer, no application message required.
- **AVL** — the device expects the explicit 4-byte AVL acknowledgement described above.

We intend to use **AVL**, because it lets the gateway acknowledge only after records are durably persisted. TCP/IP mode would acknowledge on network receipt, before

persistence, which risks silent data loss — unacceptable given this data settles billing disputes.

## 2.5 TLS is supported

TLS is implemented for the FMC1YX family (includes FMC130) **from firmware 03.27.02**. Supported versions **TLS 1.1 / 1.2**. DTLS is not fully implemented.

Teltonika's documentation states that **server-side configuration and certificate implementation is mandatory** for this to work. This needs to be designed in from the start.

Device identity remains the **IMEI**, presented at handshake. We have not confirmed whether the device presents a client certificate for mutual TLS — this is an open question with Teltonika (section 6).

---

## 3. Connection behaviour — drives ingestion sizing and cost

These four settings determine whether our fleet behaves as long-lived or short-lived connections, and how it behaves in failure. They should inform the consumption estimate directly.

**Open Link Timeout.** After the device has sent all pending records, it holds the link open for this duration. The timer refreshes each time a new record is sent. Teltonika's guidance: *"If there is a need to keep a constant link with a server, increasing the value of the parameter is needed."*

→ Default behaviour is **short-lived connections**. A persistent link is opt-in via configuration. Our current configured value is [**TECH TEAM TO CONFIRM**]. This materially changes whether load balancer cost is driven by new-connection rate or active-connection count.

**Network Ping Timeout.** The device sends a single 0xFF byte to keep the link alive and prevent the mobile operator closing an idle connection. Teltonika requires this to be **lower than Open Link Timeout**.

→ Relevant to load balancer idle timeout. If the balancer's idle timeout is shorter than our ping interval, connections will be torn down mid-session.

**Server Response Timeout.** If the server does not respond within this window, the device closes the link and **resends the same packet**.

→ Any gateway latency above this threshold produces duplicate deliveries. Our pipeline must be idempotent on record identity, not on delivery.

**Limit connection errors.** On consecutive connection failures the device backs off on a fixed schedule: no delay, 1, 1, 2, 2, 4, 6, 8, 10 minutes, then holds at 10 minutes until success.

→ This partially self-mitigates the reconnect-storm scenario after a carrier outage. Devices do not retry in lockstep. Worth factoring into burst sizing rather than assuming simultaneous reconnection.

---

## 4. Operational features relevant to the architecture

**Dual server support.** The second server can be configured as Disable, Backup, **Duplicate**, or EGTS.

In **Duplicate** mode records are sent to both servers and are deleted from device storage only when **both** confirm receipt. This gives us a safe parallel-run path — existing platform and new AWS platform receiving the same data simultaneously — with no risk of data loss during migration or POC validation. We would like this reflected in the architecture and in the cost model.

**Record ordering.** The **Sort by** setting can be Newest or Oldest. On reconnection after an outage, Oldest means historical backlog arrives before current position; Newest means live position first and backlog after. Either way the pipeline receives **out-of-order timestamps** during replay. Any time-series store selected must handle late-arriving writes without penalty.

**FOTA WEB.** Firmware management runs through Teltonika's own FOTA Web service. Connection period includes a  $\pm 2.5$ -minute randomiser to spread simultaneous device check-ins. This is a separate control plane from AWS — device firmware is not managed through AWS IoT jobs.

**Dual SIM and Auto APN.** Both supported. Auto APN selects settings from an on-device database; can be disabled where APN is pre-configured at installation.

---

## 5. What we would like SUDO to address

1. **Gateway design.** Confirm the ingestion tier that terminates TCP, completes the IMEI handshake, validates CRC-16/IBM, parses Codec 8/8E, and emits the 4-byte acknowledgement — including how it scales and how it is deployed without dropping mid-session connections.
2. **Acknowledge-after-durability.** Confirm the pipeline never returns the acknowledgement before records are durably stored, and describe the backpressure behaviour when the downstream stream or store is throttling.
3. **The IMEI handshake as an authorisation point.** The protocol lets the server reject an unknown device with 0x00 before any data is transmitted. We would like the device allow-list checked at this point. What is the expected lookup latency, and where should that registry live?

4. **TLS termination.** Where is TLS terminated, how are server certificates managed and rotated, and does terminating TLS at a load balancer preserve what the gateway needs?
  5. **Idle timeout alignment.** Confirm the load balancer idle timeout relative to our Network Ping Timeout and Open Link Timeout, so sessions are not severed mid-transaction.
  6. **Duplicate handling.** Given the device resends on Server Response Timeout, confirm the deduplication strategy and where it sits in the pipeline.
  7. **Out-of-order writes.** Confirm the chosen time-series store handles backlog replay with historical timestamps without a cost or performance penalty. Please state the write metering unit and whether small records round up.
  8. **Tenancy.** Per our covering email — pooled multi-tenancy with row-level isolation, device-to-customer as a **dated assignment**, so a client retains visibility only of the period an asset was theirs.
  9. **Cost model.** Please base the estimate on the connection behaviour in section 3, not on steady-state message counts alone.
- 

## 6. Open items — not answerable from Teltonika documentation

We are raising these with Teltonika directly. Flagging so SUDO does not design around assumptions:

| Item                                                                                                                         | Why it matters                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Mutual TLS / client certificates</b> — does the device present a client certificate, or is TLS server-authenticated only? | Determines whether cryptographic device identity is available, or whether IMEI plus network controls is the ceiling     |
| <b>CAN parameter IDs for engine hours, fuel level, RPM and fault codes</b> , per CAN adapter and per machine make            | Engine hours are our billing-dispute evidence. Base FMC130 AVL IDs are documented; CAN-sourced IDs are adapter-specific |
| <b>Whether engine hours are read from the ECU or software-estimated</b> , per machine make                                   | Commercial, not technical. An estimated value has no evidential weight in a dispute                                     |
| <b>Recommended Open Link Timeout at fleet scale</b>                                                                          | Teltonika will have deployment guidance we don't have                                                                   |
| <b>Firmware version shipping on new units</b>                                                                                | TLS requires 03.27.02+. If units ship older, they need updating before deployment                                       |
| <b>Sample Codec 8E payload captures</b>                                                                                      | Teltonika does not provide a simulator. Sample captures are the substitute — see                                        |

below

**On simulation:** Teltonika does not publish a device simulator. Two practical options for SUDO's validation work:

- We provide **live test units** and a test endpoint, so the gateway is validated against real traffic. Slower to arrange but authoritative.
- SUDO builds a **traffic generator** from the protocol specification in section 2. The Codec 8/8E frame format, IMEI handshake and acknowledgement behaviour are fully documented above and sufficient to construct valid test packets. This is the faster path and does not block on hardware.

We suggest SUDO proceed with the second while we arrange the first.

---

## 7. Sample AVL parameter IDs

From the FMC130 parameter list (firmware 03.29.00). Illustrative, not exhaustive — full list available at the Teltonika wiki.

ID

---

239

240

24

16

199

12

13

66

67

200

69

21

1 / 2

9

179

17 / 18 / 19

11

Note IDs 12 and 13 are **GNSS-estimated** fuel figures, not ECU readings. ECU-sourced fuel and engine hours arrive via CAN adapter parameters with their own IDs — see section 6.

---

*Prepared from Teltonika Telematics Wiki documentation. Items marked TECH TEAM TO CONFIRM require our device configuration before sending.*